Стохастический градиентный спуск. Часть 3

Во второй части мы узнали про важность изменения размера шага в алгоритме SGD. Продолжим улучшать стохастический градиентный спуск.

Адаптивный размер шага

Попробуем изменять размер шага отдельно для каждого параметра θ_j. Посмотрим, для чего это может быть нужно.

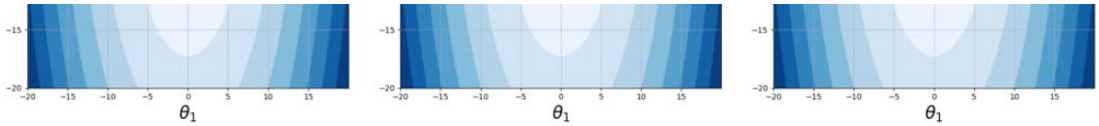
Разный наклон

Рассмотрим функцию $f(\theta_1, \theta_2) = 5\theta_1^2 + \theta_2^2$. Очевидно, что наклон по первому измерению θ_1 будет в пять раз больше, чем по второму θ_2.

$$\frac{\partial}{\partial \theta_1} f(\theta_1, \theta_2) = 10\theta_1, \quad \frac{\partial}{\partial \theta_2} f(\theta_1, \theta_2) = 2\theta_2$$

Попробуем дойти до минимума методом градиентного спуска,

- подобрав относительно большой размер шага для обоих измерений;
- применив меньший размер шага также для обоих измерений; и наконец
- выбрав больший размер шага для измерения с меньшим наклоном и наоборот меньший шаг для большего наклона.



Код для создания иллюстрации можно посмотреть в блокноте.

Сценарий 1: $\eta = 0{,}2$. Большой размер шага обеспечил уверенное продвижение по пологому второму измерению (по вертикали) и оказался избыточным для крутого наклона первого измерения (по горизонтали).

Сценарий 2: $\eta = 0{,}02$. Здесь наоборот, мы осторожно, но уверенно двигаемся по крутому первому измерению, и делаем очевидно недостаточные шаги по второму.

Сценарий 3: $\eta_1 = 0{,}02, \eta_2 = 0{,}2$. Если взять меньший шаг для большего наклона, и наоборот больший шаг для меньшего, то алгоритм быстрее сойдется к оптимуму.

Разреженные признаки

Первая иллюстрация

Рассмотрим разреженные данные (sparse data), в которых значения многих признаков равны нулю (**разреженные признаки**, sparse features).

$$X = \begin{bmatrix} 1 & 0 \\ 5 & 0 \\ 2 & 2 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 1 \\ 1{,}5 \end{bmatrix}$$

Возьмем квадратичную целевую функцию и попробуем минимизировать ошибку методом стохастического градиентного спуска. Тогда,

$$f_i(\theta) = (X_i\theta - y_i)^2,$$

где X_i — одно наблюдение (строка) матрицы X. Градиент такой функции можно найти через

$$\nabla f_i(\theta) = 2X_i^T(X_i\theta - y_i)$$

При этом будем обновлять параметры согласно

$$\theta^{k+1} := \theta^k - \eta\nabla f_i(\theta^k),$$

взяв (предположим, что случайно) сначала первое $i = 1$, потом второе $i = 2$ и наконец третье наблюдение $i = 3$. Зададим начальные веса и размер шага

$$\theta^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \eta = 0{,}01$$

Найдем градиент на первой итерации при $k = 1$ и для первого наблюдения $i = 1$.

$$\nabla f_1(\theta^1) = 2X_1^T(X_1\theta - y_1) = 2\begin{bmatrix} 1 \\ 0 \end{bmatrix}\left(\begin{bmatrix} 1 & 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix} - 2\right) =$$

$$\begin{bmatrix} 2 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} -2 \\ -2 \end{bmatrix} = \begin{bmatrix} -4 \\ 0 \end{bmatrix}$$

Поскольку второй компонент градиента равен нулю, при переходе на второй шаг обновится только первый параметр.

$$\theta^2 = \theta^1 - \eta \nabla f_1(\theta^1) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0{,}01 \begin{bmatrix} -4 \\ 0 \end{bmatrix} = \begin{bmatrix} 0{,}04 \\ 0 \end{bmatrix}$$

На следующем шаге при $k = 2$, так как $X_2 = [5 \quad 0]$, второй компонент градиента снова будет равен нулю, и соответствующий параметр снова не обновится. Обновление произойдет только на третьем шаге $k = 3$.

Проиллюстрируем этот процесс с помощью Питона и выведем значения весов и градиентов, дважды (т.е. в течение двух эпох) пройдясь по каждому из наблюдений.

```
def sparse_gradient(x, y, i, theta):
  return 2 * np.dot(x[i].T, (np.dot(x[i], theta) - y[i]))
```

```
X_s = np.array([
    [1, 0],
    [5, 0],
    [2, 2]
])

y_s = np.array([2, 1, 1.5])
```

```
eta = 0.01
theta_s = np.array([0, 0])
np.set_printoptions(precision = 2, floatmode = 'fixed')

for epoch in range(2):
  print('Epoch:', epoch+1)
  print('k', '\t', 'gradient', '\t', 'theta')
  for k in range(3):
    grad_s = sparse_gradient(X_s, y_s, k, theta_s)
    theta_s = theta_s - eta * grad_s
    print(k+1, '\t', grad_s, '\t', theta_s)
  print('--------------------------')
```

```
Epoch: 1
k        gradient        theta
1        [-4.00  0.00]   [0.04 0.00]
2        [-8.00  0.00]   [0.12 0.00]
3        [-5.04 -5.04]   [0.17 0.05]
--------------------------
Epoch: 2
k        gradient        theta
1        [-3.66  0.00]   [0.21 0.05]
2        [0.35   0.00]   [0.20 0.05]
3        [-3.97 -3.97]   [0.24 0.09]
--------------------------
```

Как мы видим, если значение признака равно нулю, соответствующий компонент градиента также равен нулю. В этой связи говорят не только про разреженные данные или признаки, но и про **разреженные градиенты** (sparse gradients).

Как следствие такой разреженности, второй параметр обновляется только один раз за эпоху.

Одновременно, поскольку размер шага для обоих признаков одинаковый, обновление весов второго признака будет «отставать», и в результате мы будем медленнее спускаться в разреженном измерении.

Это обстоятельство опять же наталкивает на мысль об использовании различных

коэффициентов скорости обучения для разреженного и плотного (dense) или заполненного признаков.

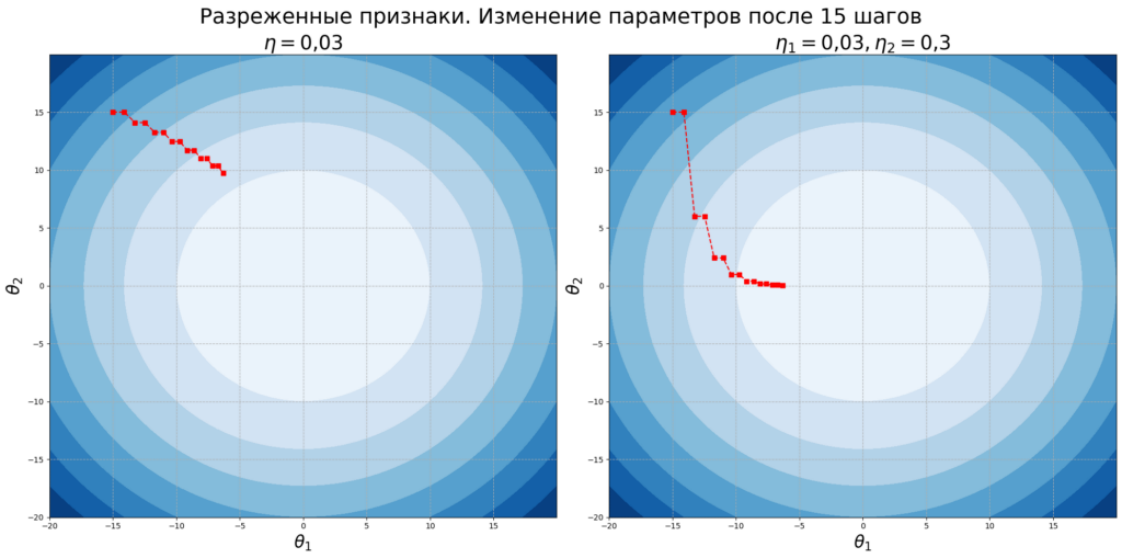
Вторая иллюстрация

Рассмотрим функцию с одинаковым наклоном по каждому из измерений

$$f(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$$

Также договоримся, что второй признак наполовину разрежен (половина наблюдений равна нулю). Имитируем это обстоятельство, обнуляя градиент при четных k шагах градиентного спуска.

Будем минимизировать f с (1) одинаковым для обоих измерений размером шага и (2) относительно бóльшим размером шага для разреженного измерения и меньшим для плотного.



Код для создания иллюстрации можно посмотреть в блокноте.

На графиках видно, что спуск по второму измерению (по вертикали) происходит реже. На четных итерациях мы двигаемся только по первому измерению (по горизонтали).

Для того чтобы компенсировать это обстоятельство, целесообразно делать больший шаг по разреженному измерению, когда представится такая возможность (т.е. градиент не будет нулевым). Именно такой подход представлен на графике справа.

Теперь возникает следующий вопрос, как подобрать удачное значение η для каждого параметра. Делать это вручную на реальных данных не представляется возможным.

AdaGrad

Алгоритм **AdaGrad** (Adaptive Gradient) решает эту задачу посредством деления размера шага *каждого* параметра θ_j на корень из суммы квадратов *соответствующих* этому параметру градиентов.

$$v^k = v^{k-1} + \nabla f_i(\theta^k)^2$$

$$\theta^{k+1} := \theta^k - \frac{\eta}{\sqrt{v^k + \varepsilon}} \odot \nabla f_i(\theta^k),$$

где ε — небольшая константа, добавляемая для вычислительной устойчивости алгоритма. Если обозначить градиент на шаге k через $g(k)$, то формулу для наглядности можно записать и так

$$G(K) = \sum_{k=1}^{K} g(k)^2$$

$$\theta^{k+1} = \theta^k - \frac{\eta}{\sqrt{G(K) + \varepsilon}} \odot g(k)$$

Обратим внимание, что возведение в квадрат и умножение $\odot$ (произведение Адамара) покомпонентные. Другими словами,

$$\begin{bmatrix} G(K)_1 \\ G(K)_2 \\ \vdots \\ G(K)_j \end{bmatrix} = \begin{bmatrix} g(k)_1^2 \\ g(k)_2^2 \\ \vdots \\ g(k)_j^2 \end{bmatrix} + \begin{bmatrix} g(k-1)_1^2 \\ g(k-1)_2^2 \\ \vdots \\ g(k-1)_j^2 \end{bmatrix} + \ldots + \begin{bmatrix} g(1)_1^2 \\ g(1)_2^2 \\ \vdots \\ g(1)_j^2 \end{bmatrix}$$

$$\begin{bmatrix} \theta_1^{k+1} \\ \theta_2^{k+1} \\ \vdots \\ \theta_j^{k+1} \end{bmatrix} = \begin{bmatrix} \theta_1^{k} \\ \theta_2^{k} \\ \vdots \\ \theta_j^{k} \end{bmatrix} - \begin{bmatrix} \frac{\eta}{\sqrt{G(K)_1+\varepsilon}} \\ \frac{\eta}{\sqrt{G(K)_2+\varepsilon}} \\ \vdots \\ \frac{\eta}{\sqrt{G(K)_j+\varepsilon}} \end{bmatrix} \odot \begin{bmatrix} g(k)_1 \\ g(k)_2 \\ \vdots \\ g(k)_j \end{bmatrix}$$

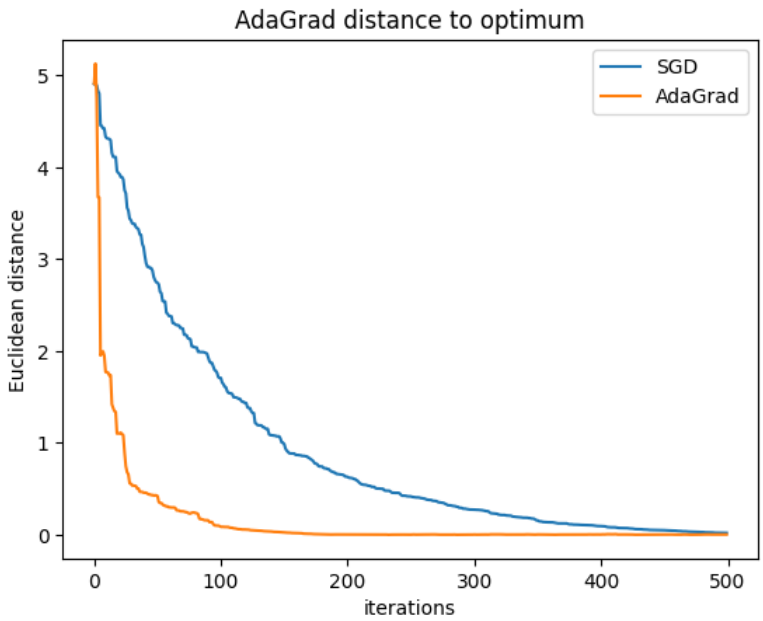
Таким образом, чем больше предыдущие градиенты (крутой склон или заполненные данные) конкретного параметра θ_j, тем сильнее уменьшается соответствующий размер шага. При этом перед началом обучения достаточно задать единственный параметр η.

```python
def fit_adagrad(x, y, t_opt, batch_size, epochs = 50, lr = 1):

  x, y = x.copy(), y.copy()
  thetas = np.zeros(x.shape[1])

  G = np.zeros(x.shape[1])
  eps = 1e-8

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      grad = gradient(x_b, y_b, thetas, n)
      G += grad ** 2
      thetas -= lr / (np.sqrt(G + eps)) * grad

  return dist_to_opt
```

```python
dist_adagrad = fit_adagrad(X, y, t_true,
```

```
                                batch_size = 1, epochs = 100,
                                lr = 1)
```

```
plt.plot(dist_sgd[:500], label = 'SGD')
plt.plot(dist_adagrad[:500], label = 'Adagrad')
plt.title('Adagrad distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance')
plt.legend()
plt.show()
```



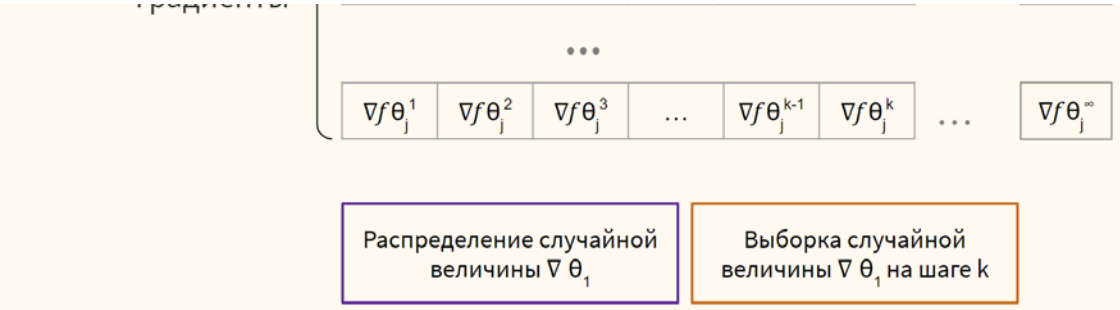
Обратите внимание, что мы взяли необычно большой начальный размер шага $\eta = 1$. На практике, кумулятивная сумма квадратов градиентов может так быстро увеличиться, что приблизит размер шага к нулю и, таким образом, замедлит обучение.

Попробуем преодолеть этот недостаток, взглянув на процесс обучения методом SGD под несколько другим углом.

Первый и второй моменты градиента

Стохастические градиенты или mini batches в процессе обучения можно рассматривать как независимые одинаково распредёленные (i.i.d.) значения некоторой случайной величины, а совокупность этих градиентов — как соответствующее распределение.

И действительно, то, какое значение градиента мы получим на шаге k случайно и зависит от того, какое i_k-ое наблюдение или какой I_k mini batch нам достанется.



Наблюдаемые же на шаге k значения градиента, то есть те градиенты, которые уже были рассчитаны на $1, 2, \ldots, k-1, k$ шагах, можно считать выборкой, по которой мы будем оценивать данное распределение.

Моменты случайной величины

Распределение можно оценить, в частности, через выборочное среднее или дисперсию. Одновременно, в более общем смысле, для численной оценки можно использовать **моменты** (moments) случайной величины.

Для случайной величины X p-ые моменты выборки размером N можно найти по формуле

$$m_p = \frac{1}{N}\sum_{n=1}^{N}(x_n - c)^p,$$

где c — константа, относительно которой рассчитывается момент m_p.

Начальный момент (raw moment) характеризует распределение вокруг нуля, т.е. $c = 0$. Первый начальный момент (first raw moment) m'_1, со степенью $p = 1$ или среднее значение (mean) выборки можно найти по формуле

$$m'_1 = \frac{1}{N}\sum_{n=1}^{N}(x_n - 0)^1$$

И действительно, если значения выборки равны $\{3, 2, 4, 1\}$, ее первый начальный момент будет равен

$$m'_1 = \frac{1}{N}\sum_{n=1}^{N}(x_n - 0)^1 = \frac{3^1}{4} + \frac{2^1}{4} + \frac{4^1}{4} + \frac{1^1}{4} =$$

$$\frac{3^1 + 2^1 + 4^1 + 1^1}{4} = 2{,}5$$

Когда мы хотим оценить разброс значений, то обращаемся ко второму моменту со степенью $p = 2$.

$$m_2 = \frac{1}{N}\sum_{n=1}^{N}(x_n - c)^2$$

При этом в большинстве случаев нас интересует разброс относительно среднего значения или центра распределения $c = m'_1$, поэтому мы обратимся не ко второму начальному, а ко второму **центральному моменту** (second central moment).

$$m_2 = \frac{1}{N}\sum_{n=1}^{N}(x_n - m_1')^2$$

Для приведенной выше выборки получим

$$m_2 = \frac{1}{N}\sum_{n=1}^{N}(x_n - 2{,}5)^2 =$$

$$\frac{(3-2{,}5)^2}{4} + \frac{(2-2{,}5)^2}{4} + \frac{(4-2{,}5)^2}{4} + \frac{(1-2{,}5)^2}{4} = 1{,}25$$

Второй центральный момент получил название дисперсия (variance). При этом ничто не мешает нам рассчитать второй начальный момент (second raw moment) m_2'. Для этого нужно положить среднее значение $m_1' = 0$.

$$m_2' = \frac{1}{N}\sum_{n=1}^{N}(x_n - 0)^2 =$$

$$\frac{(3-0)^2}{4} + \frac{(2-0)^2}{4} + \frac{(4-0)^2}{4} + \frac{(1-0)^2}{4} = 7{,}5$$

```
sample1 = np.array([3, 2, 4, 1])
m1_prime = np.mean(sample1)
m2_prime = sum([x**2 for x in sample1]) / len(sample1)
# приведем расчет дисперсии без поправки Бесселя
m2 = np.var(sample1, ddof = 0)
m1_prime, m2_prime, m2
```

```
(2.5, 7.5, 1.25)
```

Если бы значения выборки были $\{3, 2, 1, 1\}$, то

$$m_2 = 0{,}6875, \quad m_2' = 3{,}75$$

```
sample2 = np.array([3, 2, 1, 1])
m1_prime = np.mean(sample2)
m2_prime = sum([x**2 for x in sample2]) / len(sample2)
m2 = np.var(sample2, ddof = 0)
m1_prime, m2_prime, m2
```

```
(1.75, 3.75, 0.6875)
```

На практике чаще используется именно дисперсия m_2, а не второй начальный момент m_2', потому что так мы можем сравнивать разброс двух случайных величин.

```
np.random.seed(42)

# возьмем выборки распределения роста взрослых и детей
# пусть СКО роста взрослых будет меньше, чем СКО роста детей
adults = np.random.normal(180, 2, size = 1000)
children = np.random.normal(120, 3, size = 1000)
```

```
# тем не менее второй начальный момент
# роста взрослых больше
m2_prime_adults = (adults**2).mean()
m2_prime_children = (children**2).mean()
print(f'{m2_prime_adults:.2f}\t{m2_prime_children:.2f}')
```

```
32407.92    14459.99
```

```
# второй центральный момент (дисперсия) при этом
```

```
# ожидаемо больше у детей
m2_adults = adults.var(ddof = 0)
m2_children = children.var(ddof = 0)
print(f'{m2_adults:.2f}\t{m2_children:.2f}')
```

```
3.83    8.95
```

Если же сравнение не требуется, то мы можем ограничиться вторым начальным моментом случайной величины для характеристики ее изменчивости.

Приведем таблицу первого и второго начального и центрального моментов.

p	Момент	
	Начальный, m'	Центральный, m
1	m_1', среднее	m_1
2	m_2'	m_2, дисперсия

Дополнительно заметим, что первый центральный момент m_1 не слишком информативен, так как всегда равен нулю.

$$m_1 = \frac{1}{N}\sum_{n=1}^{N}(x_n - m_1')^1 = 0$$

```
m1 = sum([x-sample2.mean() for x in sample2]) / len(sample2)
m1
```

```
0.0
```

Первый момент градиента

Поскольку мы договорились считать каждый стохастический градиент или mini batch одного параметра значением случайной величины g^k, то можем оценить ее первый начальный момент по уже известным нам первым K значениям.

$$m_1' = \frac{1}{K}\sum_{k=1}^{K}(\nabla f_i(\theta_j) - 0)^1$$

Недостатком такого подхода является то, что мы задаем одинаковый вес всем предыдущим значениям градиентов (выборке). Это не оправданно, поскольку ландшафт функции потерь мог измениться, и было бы более логично отдать предполчение недавним значениям.

Для этого применим экспоненциально взвешенное скользящее среднее и найдем первый начальный момент на шаге k.

$$(m_1')^k = \beta(m_1')^{k-1} + (1-\beta)\nabla f_i(\theta_j^k)$$

Упростим нотацию, положив $(m_1') = m$,

$$m^k = \beta m^{k-1} + (1-\beta)\nabla f_i(\theta_j^k)$$

и перед нами метод SGD с инерцией.

Таким образом, стохастический градиентный спуск с инерцией использует оценку первого начального момента распределения $\nabla f_i(\theta_j)$ для каждого последующего обновления параметра θ_j.

Второй момент градиента

Оценка второго начального момента случайной величины представляет собой квадрат соответствующих значений вектора градиента.

$$m_2' = \frac{1}{K}\sum_{k=1}^{K}(\nabla f_i(\theta_j) - 0)^2,$$

то есть по сути компонент $G(K)$ алгоритма Adagrad с тем отличием, что мы не делим на количество K шагов.

RMSProp

Такое (экспоненциальное) усреднение вводится алгоритмом **RMSprop** (Root Mean Squared Propagation).

$$v^k = \beta v^{k-1} + (1-\beta)g(k)^2$$

$$\theta^{k+1} = \theta^k - \frac{\eta}{\sqrt{v^k + \varepsilon}} \odot g(k)$$

Так же как и в случае с SGD с инерцией мы обращаем больше внимания на недавние градиенты, только здесь в зависимости от этого корректируем размер шага.

```
def fit_rmsprop(x, y, t_opt, batch_size,
                epochs = 50, lr = 0.01, beta = 0.9):

  x, y = x.copy(), y.copy()
  thetas = np.zeros(x.shape[1])

  vk = np.zeros(x.shape[1])
  eps = 1e-8

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      grad = gradient(x_b, y_b, thetas, n)
      vk = beta * vk + (1 - beta) * np.square(grad)
      thetas -= (lr / np.sqrt(vk + eps)) * grad

  return dist_to_opt
```

Алгоритм Adam

Алгоритм **Adam** (Adaptive Moment Estimation, т.е. «адаптивная оценка моментов [случайной величины]») объединяет многие из высказанных ранее идей улучшения стохастического градиентного спуска.

В частности, алгоритм учитывает инерцию (первый начальный момент)

$$m^k = \beta_1 m^{k-1} + (1 - \beta_1) g(k),$$

а также изменчивость градиента (второй начальный момент)

$$v^k = \beta_2 v^{k-1} + (1 - \beta_2) g(k)^2,$$

вводит корректировку экспоненциального сглаживания как первого $\hat{m}^k$, так и второго $\hat{v}^k$ моментов

$$\hat{m}^k = \frac{m^k}{1 - \beta_1^k}, \quad \hat{v}^k = \frac{v^k}{1 - \beta_2^k}$$

и наконец обновляет веса, корректируя направление и размер шага,

$$\theta^{k+1} := \theta^k - \eta \frac{\hat{m}^k}{\sqrt{\hat{v}^k} + \varepsilon}.$$

Таким образом, алгоритм снабжен тремя гиперпараметрами:

- размером шага η; а также
- двумя коэффициентами затухания, которые авторы алгоритма рекомендуют по умолчанию установить на уровне $\beta_1 = 0{,}9, \beta_2 = 0{,}999$.

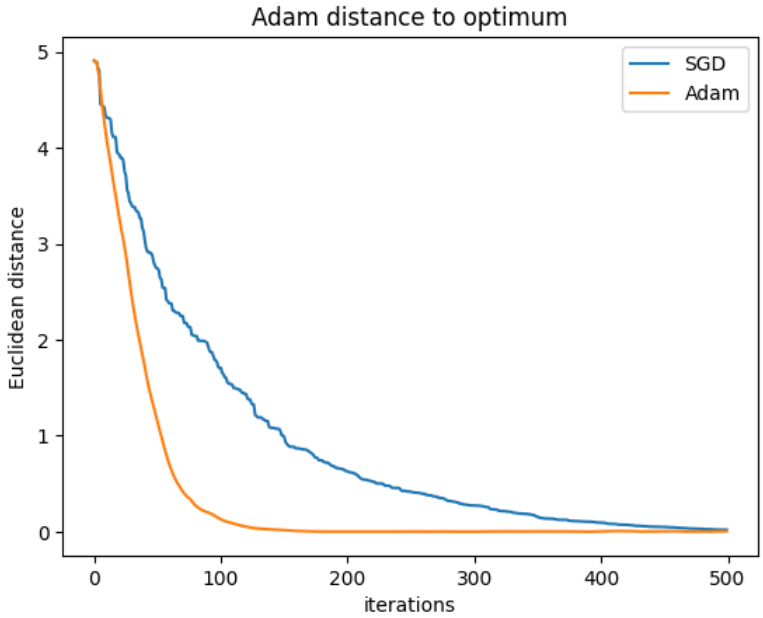
Опять же, умножение векторов поэлементное.

```
def fit_adam(x, y, t_opt, batch_size, epochs = 50,
             lr = 0.1,
             beta1 = 0.9, beta2 = 0.999):

  x, y = x.copy(), y.copy()
  thetas = np.zeros(x.shape[1])

  mk = np.zeros(x.shape[1])
  vk = np.zeros(x.shape[1])
  eps = 1e-8
  k = 1

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      grad = gradient(x_b, y_b, thetas, n)

      mk = beta1 * mk + (1 - beta1) * grad
      vk = beta2 * vk + (1 - beta2) * np.square(grad)

      mhat = mk / (1 - beta1 ** k )
```

```
        vhat = vk / (1 - beta2 ** k )

        thetas -= lr * mhat/(np.sqrt(vhat) + eps)
        k += 1

    return dist_to_opt
```

```
dist_adam = fit_adam(X, y, t_true, batch_size = 1)
```

```
plt.plot(dist_sgd[:500], label = 'SGD')
plt.plot(dist_adam[:500], label = 'Adam')
plt.title('Adam distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance')
plt.legend()
plt.show()
```



Рассмотрим некоторые идеи развития этого алгоритма.

AdaMax

Интересно, что второй начальный момент v^k можно представить как экспоненциально взвешенную (до извлечения корня) L_2 норму вектора значений текущего и прошлых градиентов $L_2 = (v^k)^{\frac{1}{2}}$.

Тогда в общем виде L_p норму $(v^k)^{\frac{1}{p}}$ можно записать как

$$||\mathbf{x}||_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}} \longrightarrow (v^k)^{\frac{1}{p}} = \left(\beta_2^p v^{k-1} + (1-\beta_2^p)|g(k)|^p\right)^{\frac{1}{p}}$$

Обратите внимание, $\beta_2 \to \beta_2^p$. Тогда по формуле экспоненциально взвешенного скользящего среднего

$$(v^k)^{\frac{1}{p}} = \left((1-\beta_2^p)\sum_{i=1}^{k} \beta_2^{p(k-i)} \cdot |g_i|^p\right)^{\frac{1}{p}}$$

Положив $u^k = \lim_{p\to\infty}(v^k)^{\frac{1}{p}}$ (расстояние Чебышёва), получим

$$\begin{aligned} u^k = \quad & \lim_{p\to\infty}(v^k)^{\frac{1}{p}} = \\ & \lim_{p\to\infty}\left((1-\beta_2^p)\sum_{i=1}^k \beta_2^{p(k-i)}\cdot|g_i|^p\right)^{\frac{1}{p}} = \\ & \lim_{p\to\infty}\underbrace{(1-\beta_2^p)^{\frac{1}{p}}}_{1}\left(\sum_{i=1}^k \beta_2^{p(k-i)}\cdot|g_i|^p\right)^{\frac{1}{p}} = \\ & \lim_{p\to\infty}\left(\sum_{i=1}^k \beta_2^{p(k-i)}\cdot|g_i|^p\right)^{\frac{1}{p}} \end{aligned}$$

Расстояние Чебышёва можно найти через максимальное абсолютное значение координат вектора

$$\lim_{p\to\infty}\left(\sum_{i=1}^n |x_i|^p\right)^{\frac{1}{p}} = \max_{i=1}^n |x_i|$$

Покажем это неформально. Пусть дан вектор $\mathbf{x} = [1, 2, 3]$. Тогда по формуле нормы вектора

$$||\mathbf{x}||_p = (1^p + 2^p + 3^p)^{\frac{1}{p}} = 3\cdot\left(\left(\frac{1}{3}\right)^p + \left(\frac{2}{3}\right)^p + 1\right)^{1/p}$$

При $p\to\infty$; $\left(\frac{1}{3}\right)^p, \left(\frac{2}{3}\right)^p \to 0$, а значит $||\mathbf{x}||_p \to 3$.

Приведем более формальное доказательство. Пусть $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ — вектор с конечным числом элементов.

Допустим (поскольку $\mathbf{x}$ конечен), $0 \le x_1 \le x_2 \le \cdots \le x_n$ так, что $x_n = \max_i |x_i|$.

При $x_n = 0$,

$$\forall p \ge 1, \qquad 0 = ||x||_p \xrightarrow[p\to\infty]{} 0 = x_n$$

Если $x_n > 0$,

$$\forall p \ge 1, \qquad \|x\|_p = x_n\left(\sum_{k=1}^n \left(\frac{x_k}{x_n}\right)^p\right)^{\frac{1}{p}}$$

Поскольку $\forall k, 1 \le k \le n$ выполняется $0 \le \frac{x_k}{x_n} \le 1$, получаем, что $||x||_p$ имеет следующие нижнюю и верхнюю границу

$$\forall p \ge 1, \qquad x_n \cdot 1^{\frac{1}{p}} \le ||x||_p \le x_n \cdot n^{\frac{1}{p}}$$

В силу того, что $\lim_{p\to\infty} 1^{\frac{1}{p}} = 1$ и $\lim_{p\to\infty} n^{\frac{1}{p}} = 1$, согласно squeeze theorem получаем, что

$$\lim_{p\to\infty} ||x||_p = x_n = \max_i |x_i|.$$

В случае значений градиента получим следующие явную и рекуррентную формы

$$
\begin{aligned}
u^k &= \max(\beta_2^{k-1}|g(1)|, \beta_2^{k-2}|g(2)|, \ldots, \beta_2|g(k-1)|, |g(k)|) \\
&= \max(\beta_2 \cdot u^{k-1}, |g(k)|)
\end{aligned}
$$

Такой алгоритм, использующий для масштабирования размера шага наибольший по модулю экспоненциально сглаженный градиент, по понятной причине называется **AdaMax**.

Найдем первый и второй моменты.

$$
\begin{aligned}
m^k &= \beta_1 m^{k-1} + (1-\beta_1) g(k) \\
u^k &= \beta_2^\infty v^{k-1} + (1-\beta_2^\infty)|g(k)|^\infty \\
&= \max(\beta_2 \cdot v^{k-1}, |g(k)|)
\end{aligned}
$$

Сделаем коррекцию смещения первого момента относительно размера шага.

$$
\hat{\eta} = \frac{\eta}{(1-\beta_1^k)}
$$

В силу того, что u_0 (в данном случае равное нулю) не влияет на расчет последующих u^k значений, корректировка смещения этого компонента не требуется. Наконец обновим веса.

$$
\theta^{k+1} := \theta^k - \hat{\eta}\frac{m^k}{u^k}
$$

```
def fit_adamax(x, y, t_opt, batch_size, epochs = 50,
               lr = 0.01,
               beta1 = 0.9, beta2 = 0.999):

  x, y = x.copy(), y.copy()
  thetas = np.zeros(x.shape[1])

  mk = np.zeros(x.shape[1])
  uk = np.zeros(x.shape[1])
  k = 1

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      grad = gradient(x_b, y_b, thetas, n)

      mk = beta1 * mk + (1 - beta1) * grad
      uk = np.max([beta2 * uk, np.abs(grad)], axis=0)

      lr_hat = lr / (1 - beta2 ** k)
      k += 1

      thetas -= lr_hat * mk/uk

  return dist_to_opt
```

AdamW

Рассмотрим применение регуляризующих слагаемых к упомянутым алгоритмам оптимизации.

В случае L2 регуляризации речь идет о

$$\hat{f}(\theta) = f(\theta) + \frac{\lambda'}{2}||\theta||_2^2,$$

где $\lambda' = \frac{\lambda}{\eta}$, λ — коэффициент регуляризации, а η — размер шага. Как следствие, производная будет иметь вид

$$\nabla \hat{f}(\theta) = \nabla f(\theta) + \lambda' \theta$$

Обновление параметров при этом будет происходить согласно

$$\begin{aligned} \theta^{k+1} &:= \theta^k - \eta \nabla \hat{f}(\theta) \\ &:= \theta^k - \eta(\nabla f(\theta) + \lambda' \theta) \\ &:= \theta^k - \eta \nabla f(\theta) - \lambda \theta, \end{aligned}$$

Как мы видим, обновленные параметры дополнительно уменьшаются на величину $\lambda\theta$, в силу чего регуляризацию еще называют убыванием весов (weight decay), а λ соответственно скоростью убывания (decay rate).

При этом если для классического SGD L2 регуляризация (добавление $\lambda'\theta$ к функции потерь) и weight decay (уменьшение весов на $\lambda\theta$) — это одно и то же (выше мы это показали), то в случае более сложных оптимизаторов это не так.

Рассмотрим алгоритм Adam и применим L2 регуляризацию. Найдем первый момент m^k

$$m^k = \beta_1 m^{k-1} + (1 - \beta_1)(\nabla f(\theta) + \lambda\theta)$$

и подставим его в правило обновления весов

$$\begin{aligned} \theta^{k+1} &:= \theta^k - \eta \left(\beta_1 m^{k-1} + (1 - \beta_1)(\nabla f(\theta) + \lambda' \theta)\right) \\ &:= \theta^k - \eta \beta_1 m^{k-1} - \eta(1 - \beta_1)(\nabla f(\theta) + \lambda' \theta) \\ &:= \theta^k - \eta \beta_1 m^{k-1} - \eta(1 - \beta_1)\nabla f(\theta) - \lambda(1 - \beta_1)\theta \end{aligned}$$

В таком виде регуляризация участвует в расчете экспоненциального среднего и, как следствие, не равна убыванию весов. Аналогичная сложность возникает при расчете второго момента v^k.

Авторы алгоритма **AdamW** предлагают использовать «отсоединенное» убывание весов (decoupled weight decay). Функция потерь и соответственно градиент рассчитываются без учета регуляризации

$$g(k) = \nabla f(\theta^k) + \cancel{\lambda \theta^k}$$

Расчет первого и второго момента аналогичен алгоритму Adam

$$m^k = \beta_1 m^{k-1} + (1 - \beta_1) g(k)$$

$$v^k = \beta_2 v^{k-1} + (1 - \beta_2) g(k)^2$$

$$\hat{m}^k = \frac{m^k}{1 - \beta_1^k}, \quad \hat{v}^k = \frac{v^k}{1 - \beta_2^k}$$

и только при обновлении весов появляется регуляризация

$$\theta^{k+1} := \theta^k - \eta \frac{\hat{m}^k}{\sqrt{\hat{v}^k} + \varepsilon} - \lambda \theta^k$$

Исследование авторов AdamW показывает, что результаты алгоритмов с отсоединенным убыванием весов на тестовых данных на 15% выше, чем у L2 регуляризации.

Сравните код ниже.

```
# L2 регуляризация
# reg - коэффициент регуляризации
grad = gradient(x_b, y_b, thetas, n) + reg * thetas

mk = beta1 * mk + (1 - beta1) * grad
vk = beta2 * vk + (1 - beta2) * np.square(grad)

mhat = mk / (1 - beta1 ** k)
vhat = vk / (1 - beta2 ** k)

thetas -= lr * mhat/(np.sqrt(vhat) + eps)
```

```
grad = gradient(x_b, y_b, thetas, n)

mk = beta1 * mk + (1 - beta1) * grad
vk = beta2 * vk + (1 - beta2) * np.square(grad)

mhat = mk / (1 - beta1 ** k)
vhat = vk / (1 - beta2 ** k)

# weight decay
thetas -= (lr * mhat/(np.sqrt(vhat) + eps) + reg * thetas)
```

Помимо AdamW decoupled weight decay очевидно можно применить и к другим оптимизаторам, например, к SGDm, AdaGrad или RMSProp.

В дополнительных материалах мы реализуем алгоритм Adam для нейросети, а также попрактикуемся в создании модели в библиотеке Keras.

Подведем итог

В третьей части мы рассмотрели проблему разного наклона целевой функции и разреженных градиентов и, как следствие, идею адаптивного размера шага (алгоритмы AdaGrad и RMSProp).

Алгоритм Adam объединил принцип инерции и адаптивного размера шага с коррекцией экспоненциального сглаживания.

В методе AdaMax L_2 норма вектора градиентов заменена на расстояние Чебышёва. И наконец метод AdamW позволил применить такую регуляризацию весов, которая не влияет на расчет первого и второго моментов алгоритма.

Список литературы

- Adaptive Subgradient Methods for Online Learning and Stochastic Optimization (AdaGrad), Duchi et al., 2011
- COURSERA: Neural Networks for Machine Learning (RMSProp), 4, 26-31, G Hinton, 2012
- Adam: A Method for Stochastic Optimization (Adam, AdaMax), DP Kingma, 2015
- Decoupled Weight Decay Regularization (AdamW), Loshchilov & Hutter, 2019